API ESCOLA

Alunos, Cursos e Matrículas

Desenvolvimento de uma API REST com banco de dados SQLite e hospedagem de ponta a ponta em uma máquina virtual na Microsoft Azure

---

Linguagem: Python 3.12 + FastAPI

Banco de dados: SQLite 3

Nuvem: Microsoft Azure – Central US, VM Standard_B2pts_v2 (Arm64)

API publicada: http://52.230.164.119/docs

Repositório: https://github.com/RenesFernandes7366/api-alunos

Data: Outubro de 2026

Sumário

1. Introdução e objetivo.....3
1.1 Entregáveis.....3
2. Visão geral da solução.....4
2.1 Por que essa arquitetura.....4
3. Tecnologias e linguagem de programação.....6
4. Banco de dados SQLite.....7
4.1 Modelo entidade-relacionamento.....7
4.2 Regras garantidas pelo banco.....8
5. A API.....9
5.1 Estrutura do código.....9
5.2 Rotas (endpoints).....9
5.3 Códigos de resposta HTTP.....10
5.4 Caminho de uma requisição.....10
5.5 Exemplo de requisição e resposta.....10
6. Testes e evidências (prints de tela).....11
6.1 Registro das evidências.....11
6.2 Testes automatizados (pytest).....11
6.3 Teste de ponta a ponta local.....12
6.4 Documentação interativa (Swagger UI).....12
6.5 API publicada na Azure.....15
7. Elementos de nuvem (Microsoft Azure).....17
7.1 Máquina virtual e SKU.....18
7.2 Armazenamento.....19
7.3 Rede e segurança.....20
7.4 Configuração da VM (cloud-init e instalar_na_vm.sh).....21
8. Passo a passo do deploy.....23
8.1 Rodar e testar localmente.....23
8.2 Publicar no GitHub.....23
8.3 Criar a infraestrutura na Azure.....23
8.4 Configurar a VM (quando necessário).....23
8.5 Validar e remover.....24
8.6 Problemas encontrados e soluções.....24
9. Repositório no GitHub.....25
10. Melhorias futuras e conclusão.....26
10.1 Melhorias possíveis.....26
10.2 Conclusão.....26

1. Introdução e objetivo

Este documento descreve, passo a passo, o desenvolvimento e a publicação de uma **API REST básica** para o cadastro de **alunos**, **cursos** e **matrículas**. A API grava os dados em um banco **SQLite** e é hospedada de ponta a ponta em uma **máquina virtual Linux na Microsoft Azure**, ficando acessível pela internet através do IP público **52.230.164.119**.

O trabalho cobre todas as etapas do ciclo: escrita do código, testes automatizados, versionamento no GitHub, criação da infraestrutura na nuvem, configuração do servidor e validação final com chamadas HTTP reais.

1.1 Entregáveis

- **Código-fonte** da API em Python (FastAPI) com banco SQLite.
- **Repositório no GitHub** com código, testes, scripts de deploy e documentação.
- **Esquemas visuais**: arquitetura de nuvem, modelo entidade-relacionamento, fluxo de uma requisição e fluxo de deploy.
- **Prints de tela** dos testes locais, da API publicada na Azure e do portal da Azure, com data e hora.
- **Explicação dos elementos de nuvem**: VM e SKU, armazenamento, rede, segurança e sistema operacional.
- Este documento, em **Word** e **PDF**.

2. Visão geral da solução

A solução segue uma arquitetura simples de três camadas dentro de uma única máquina virtual. O usuário acessa o **IP público** da VM pela porta 80. O **Network Security Group (NSG)** permite apenas as portas 22 (SSH, administração) e 80 (HTTP). Dentro da VM, o **Nginx** recebe a requisição e a repassa para o **Uvicorn**, servidor que executa a aplicação **FastAPI**. A aplicação lê e grava no arquivo **SQLite**, que fica no disco gerenciado (Managed Disk) da VM.

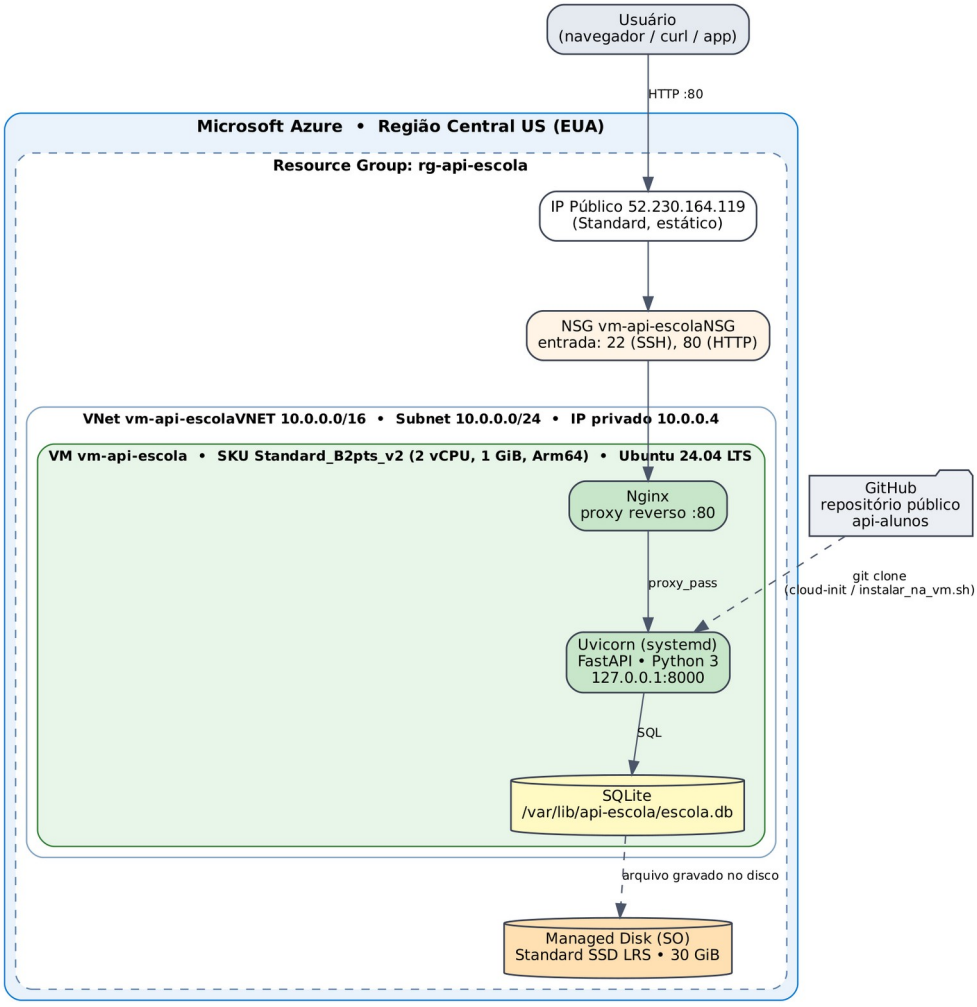


Figura 1 – *Arquitetura da solução na Azure*

2.1 Por que essa arquitetura

- **Simplicidade:** uma única VM concentra servidor web, aplicação e banco. É o modelo mais didático para entender cada peça da nuvem.
- **Custo baixo:** a SKU `Standard_B2pts_v2` é uma das menores da Azure e está entre os tamanhos incluídos nas horas gratuitas da conta free para novos clientes.
- **SQLite dispensa servidor de banco:** o banco é um arquivo no disco, sem instalação ou serviço extra. É adequado a uma API básica com uma única instância.

- **Nginx na frente:** protege o Uvicorn (que escuta só em 127.0.0.1), permite HTTPS no futuro e é o padrão de mercado para publicar aplicações Python.

3. Tecnologias e linguagem de programação

A linguagem escolhida foi **Python**, pela sintaxe clara, pelo ecossistema maduro para APIs e por já vir instalada no Ubuntu. O framework **FastAPI** foi escolhido porque valida os dados automaticamente e gera a documentação interativa (Swagger) sem código extra.

Camada	Tecnologia	Função no projeto
Linguagem	Python 3.12	Linguagem de todo o back-end (versão padrão do Ubuntu 24.04).
Framework web	FastAPI	Define as rotas HTTP, converte JSON e gera a documentação OpenAPI/Swagger.
Servidor de aplicação	Uvicorn (ASGI)	Executa a aplicação FastAPI e atende as requisições na porta 8000.
Validação	Pydantic v2	Valida tipos, tamanhos, e-mail e datas; devolve erro 422 se algo estiver inválido.
Banco de dados	SQLite 3	Banco relacional em arquivo único (`escola.db`), acessado pelo módulo `sqlite3` do Python.
Proxy reverso	Nginx	Recebe o tráfego público na porta 80 e repassa ao Uvicorn.
Gerência de serviço	systemd	Mantém a API rodando, reinicia em caso de falha e inicia no boot.
Provisionamento	Azure CLI + cloud-init / Run Command	Cria os recursos na Azure e configura a VM (Nginx, Python, serviço da API).
Testes	pytest + httpx	8 testes automatizados cobrindo CRUD, validações e regras de negócio.
Versionamento	Git + GitHub	Histórico do código e fonte de onde a VM baixa a aplicação.

4. Banco de dados SQLite

O **SQLite** é um banco de dados relacional que guarda tudo em **um único arquivo**. Não há processo servidor: a própria aplicação abre o arquivo e executa comandos SQL. Na VM, o arquivo fica em `/var/lib/api-escola/escola.db`; o caminho é definido pela variável de ambiente `DB_PATH`.

4.1 Modelo entidade-relacionamento

O modelo tem três tabelas. Um aluno pode se matricular em vários cursos, e um curso pode ter vários alunos, um relacionamento **muitos-para-muitos** resolvido pela tabela associativa `matriculas`.

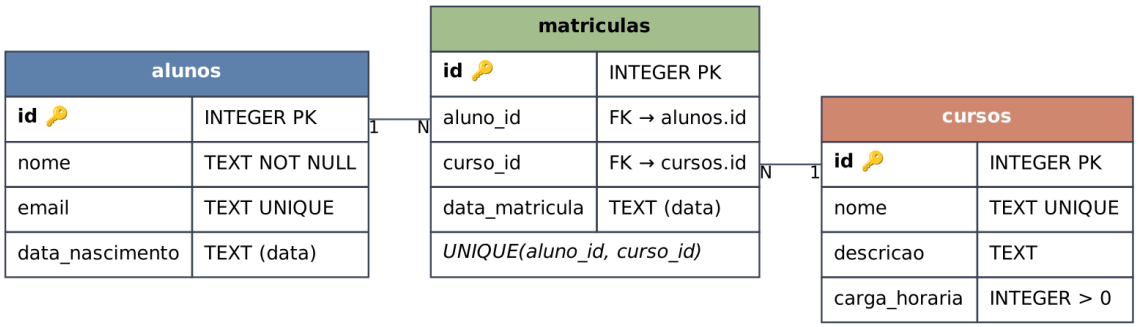


Figura 2 – *Modelo entidade-relacionamento (ER)*

Tabela	Campo	Tipo / restrição	Descrição
alunos	id	INTEGER PK AUTOINCREMENT	Identificador do aluno
alunos	nome	TEXT NOT NULL	Nome completo
alunos	email	TEXT NOT NULL UNIQUE	E-mail, sem repetição
alunos	data_nascimento	TEXT (AAAA-MM-DD)	Opcional
cursos	id	INTEGER PK AUTOINCREMENT	Identificador do curso
cursos	nome	TEXT NOT NULL UNIQUE	Nome do curso, sem repetição
cursos	descricao	TEXT	Opcional
cursos	carga_horaria	INTEGER CHECK (> 0)	Horas do curso
matriculas	id	INTEGER PK AUTOINCREMENT	Identificador da matrícula
matriculas	aluno_id	FK → alunos.id ON DELETE CASCADE	Aluno matriculado
matriculas	curso_id	FK → cursos.id ON DELETE CASCADE	Curso da matrícula

Tabela	Campo	Tipo / restrição	Descrição
matriculas	data_matricula	TEXT DEFAULT date('now')	Data automática (UTC)

4.2 Regras garantidas pelo banco

- **UNIQUE(aluno_id, curso_id):** o mesmo aluno não pode se matricular duas vezes no mesmo curso.
- **ON DELETE CASCADE:** ao remover um aluno ou curso, as matrículas dele são removidas junto, sem deixar registros órfãos.
- **PRAGMA foreign_keys = ON:** ativado em toda conexão, porque o SQLite não valida chaves estrangeiras por padrão.
- **Criação automática:** as tabelas são criadas na primeira execução (`CREATE TABLE IF NOT EXISTS`), e recriadas se o arquivo for apagado.

Trecho do esquema (app/database.py)

```
CREATE TABLE IF NOT EXISTS matriculas (
    id             INTEGER PRIMARY KEY AUTOINCREMENT,
    aluno_id       INTEGER NOT NULL REFERENCES alunos(id) ON DELETE CASCADE,
    curso_id       INTEGER NOT NULL REFERENCES cursos(id) ON DELETE CASCADE,
    data_matricula TEXT    NOT NULL DEFAULT (date('now')),
    UNIQUE (aluno_id, curso_id)
);
```

5. A API

5.1 Estrutura do código

```
api-alunos/
├── app/
│   ├── main.py           # rotas da API (FastAPI)
│   ├── database.py       # conexão e criação das tabelas SQLite
│   └── schemas.py        # validação dos dados (Pydantic)
├── tests/test_api.py     # 8 testes automatizados (pytest)
├── scripts/smoke_test.sh # teste HTTP de ponta a ponta
├── deploy/
│   ├── azure_criar_vm.sh # cria a infraestrutura na Azure
│   ├── cloud-init.yaml   # configura a VM no primeiro boot
│   └── atualizar_vm.sh   # atualiza a API após novo push
├── docs/                 # diagramas e prints
├── requirements.txt      # dependências de produção
└── README.md
```

- **database.py** abre a conexão com o SQLite, ativa as chaves estrangeiras, faz commit/rollback automaticamente e fecha a conexão ao final (gerenciador de contexto `with`).
- **schemas.py** define os modelos de entrada (`AlunoIn`, `CursoIn`, `MatriculaIn`) e de saída. O Pydantic rejeita, por exemplo, e-mail inválido ou carga horária zero.
- **main.py** declara as rotas. Todas as consultas usam **parâmetros** (`?`) em vez de concatenar texto, o que impede **SQL Injection**.

5.2 Rotas (endpoints)

Método	Rota	O que faz	Respostas
GET	/health	Verifica se a API e o banco respondem	200
GET	/cursos	Lista todos os cursos	200
POST	/cursos	Cria um curso	201, 409, 422
GET	/cursos/{id}	Busca um curso	200, 404
PUT	/cursos/{id}	Atualiza um curso	200, 404, 409, 422
DELETE	/cursos/{id}	Remove um curso (e suas matrículas)	204, 404
GET	/alunos	Lista todos os alunos	200
POST	/alunos	Cria um aluno	201, 409, 422
GET	/alunos/{id}	Busca um aluno	200, 404
PUT	/alunos/{id}	Atualiza um aluno	200, 404, 409, 422
DELETE	/alunos/{id}	Remove um aluno (e suas matrículas)	204, 404

Método	Rota	O que faz	Respostas
GET	/alunos/{id}/cursos	Lista os cursos em que o aluno está matriculado	200, 404
GET	/matriculas	Lista matrículas com nome do aluno e do curso	200
POST	/matriculas	Matricula um aluno em um curso	201, 404, 409, 422
DELETE	/matriculas/{id}	Cancela uma matrícula	204, 404

5.3 Códigos de resposta HTTP

Código	Significado	Quando ocorre
200 OK	Sucesso	Consultas e atualizações bem-sucedidas
201 Created	Criado	Novo aluno, curso ou matrícula
204 No Content	Removido	Exclusão bem-sucedida
404 Not Found	Não encontrado	ID inexistente
409 Conflict	Conflito	E-mail, nome de curso ou matrícula repetidos
422 Unprocessable	Dados inválidos	JSON fora do formato (ex.: e-mail inválido)

5.4 Caminho de uma requisição

A figura abaixo mostra o que acontece quando um cliente envia `POST /matriculas`. Se o JSON for inválido, o Pydantic devolve 422 antes de tocar no banco. Se o aluno ou curso não existir, a API devolve 404; se a matrícula já existir, 409.

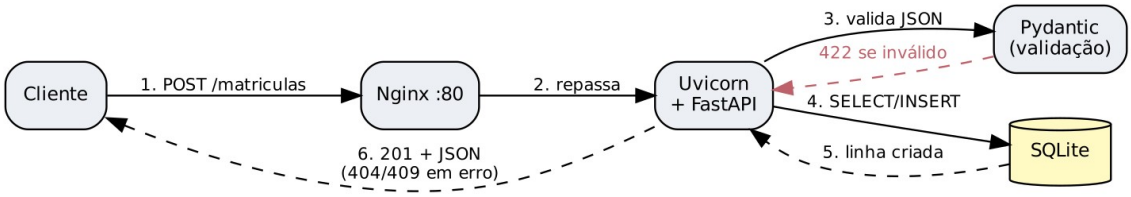


Figura 3 – *Fluxo de uma requisição dentro da VM*

5.5 Exemplo de requisição e resposta

```
curl -X POST http://52.230.164.119/matriculas \
     -H "Content-Type: application/json" \
     -d '{"aluno_id": 1, "curso_id": 1}'

HTTP/1.1 201 Created
{"aluno_id":1,"curso_id":1,"id":1,"data_matricula":"2026-10-26",
 "aluno_nome":"Maria Silva","curso_nome":"Computação em Nuvem"}
```

6. Testes e evidências (prints de tela)

A API foi testada em duas etapas. Primeiro **localmente**, com **testes automatizados** (pytest), que usam um banco SQLite temporário e isolado para cada teste, e com um **teste de ponta a ponta** (`smoke_test.sh`), que faz chamadas HTTP reais com `curl`. Depois, **na nuvem**, o mesmo script foi executado contra o IP público da VM na Azure.

6.1 Registro das evidências

Horários no fuso de Brasília (UTC−3). Os prints de 01 a 06 foram gerados automaticamente com a API rodando localmente. Os prints de 07 a 14 foram capturados no navegador e no portal da Azure, e o horário indicado é o aproximado da captura.

Print	Conteúdo	Ambiente	Data e hora
01	Swagger – visão geral das rotas	Local	25/10/2026 22:57:26
02	Swagger – cadastro de aluno (201)	Local	25/10/2026 22:57:27
03	Swagger – listagem de matrículas	Local	25/10/2026 22:57:28
04	Rota /health no navegador	Local	25/10/2026 22:57:28
05	pytest – 8 testes aprovados	Local	25/10/2026 22:57:28
06	smoke_test.sh – chamadas HTTP	Local	25/10/2026 22:57:29
07	Swagger servido pelo IP público	Azure	26/10/2026 ≈ 00:33
08	smoke_test.sh contra o IP público	Azure	26/10/2026 ≈ 00:35
09	Resource Group e recursos criados	Portal Azure	26/10/2026 ≈ 00:37
10	VM – visão geral (SO, Arm64, rede)	Portal Azure	26/10/2026 ≈ 00:39
11	VM – imagem de origem e disco	Portal Azure	26/10/2026 ≈ 00:39
12	VM – tamanho (SKU)	Portal Azure	26/10/2026 ≈ 00:40
13	VM – configurações de rede (NSG)	Portal Azure	26/10/2026 ≈ 00:41
14	NSG – regras de segurança de entrada	Portal Azure	26/10/2026 ≈ 00:49

Nos prints 02 e 04 e na saída do smoke test, a API mostra a hora em **UTC** (3 horas a mais que Brasília), que é o padrão de servidores.

6.2 Testes automatizados (pytest)

Os 8 testes cobrem: health check, CRUD completo de cursos e alunos, nome de curso duplicado (409), carga horária inválida (422), e-mail inválido e duplicado, fluxo completo de matrícula (incluindo matrícula repetida e exclusão em cascata) e matrícula com IDs inexistentes (404). **Resultado: 8 de 8 aprovados.**

```
pytest -v — testes automatizados

$ pytest -v
============================= test session starts ==============================
platform linux -- Python 3.13.17, pytest-9.1.1, pluggy-1.7.0 -- /usr/local/bin/python3
cachedir: .pytest_cache
rootdir: /home/claude/api-alunos-cursos
plugins: platformdirs-4.12.4, anyio-4.15.1
collecting ... collected 8 items

tests/test_api.py::test_health PASSED                                    [ 12%]
tests/test_api.py::test_crud_curso PASSED                                [ 25%]
tests/test_api.py::test_curso_duplicado_retorna_409 PASSED               [ 37%]
tests/test_api.py::test_validacao_carga_horaria PASSED                   [ 50%]
tests/test_api.py::test_crud_aluno PASSED                                [ 62%]
tests/test_api.py::test_email_invalido_e_duplicado PASSED                [ 75%]
tests/test_api.py::test_matricula_fluxo_completo PASSED                  [ 87%]
tests/test_api.py::test_matricula_com_ids_inexistentes PASSED            [100%]

============================== 8 passed in 0.38s ===============================
```

Figura 4 – *Execução dos testes automatizados – 8 testes aprovados*

6.3 Teste de ponta a ponta local

O script cria dois cursos, dois alunos e três matrículas, consulta os cursos de um aluno e confirma os erros esperados: matrícula duplicada (409) e aluno inexistente (404).

```
./scripts/smoke_test.sh — chamadas HTTP reais

$ curl http://127.0.0.1:8000/health
{"status":"ok","banco":"sqlite","horario_utc":"2026-10-26T01:56:44.462884+00:00"}

$ curl -X POST http://127.0.0.1:8000/cursos
{"nome":"Computação em Nuvem","descricao":"Fundamentos de Azure","carga_horaria":60,"id":1}

$ curl -X POST http://127.0.0.1:8000/cursos
{"nome":"Banco de Dados","descricao":"SQL e modelagem","carga_horaria":80,"id":2}

$ curl -X POST http://127.0.0.1:8000/alunos
{"nome":"Maria Silva","email":"maria@email.com","data_nascimento":"2001-05-14","id":1}

$ curl -X POST http://127.0.0.1:8000/alunos
{"nome":"João Souza","email":"joao@email.com","data_nascimento":null,"id":2}

$ curl -X POST http://127.0.0.1:8000/matriculas
{"aluno_id":1,"curso_id":1,"id":1,"data_matricula":"2026-10-26","aluno_nome":"Maria Silva","curso_nome":"Computação em Nuvem"}
{"aluno_id":1,"curso_id":2,"id":2,"data_matricula":"2026-10-26","aluno_nome":"Maria Silva","curso_nome":"Banco de Dados"}
{"aluno_id":2,"curso_id":1,"id":3,"data_matricula":"2026-10-26","aluno_nome":"João Souza","curso_nome":"Computação em Nuvem"}

$ curl http://127.0.0.1:8000/alunos/1/cursos
[{"nome":"Computação em Nuvem","descricao":"Fundamentos de Azure","carga_horaria":60,"id":1},{"nome":"Banco de Dados","descricao":"SQL e modelage
m","carga_horaria":80,"id":2}]

$ curl -X POST http://127.0.0.1:8000/matriculas  (duplicada -> 409)
{"detail":"Aluno já matriculado neste curso"}  [HTTP 409]

$ curl http://127.0.0.1:8000/alunos/999  (inexistente -> 404)
{"detail":"Aluno 999 não encontrado(a)"}  [HTTP 404]
```

Figura 5 – *Saída do smoke_test.sh com chamadas reais à API local*

6.4 Documentação interativa (Swagger UI)

O FastAPI gera automaticamente a página `/docs`, que lista todas as rotas e permite testá-las direto no navegador pelo botão **Try it out**.



Figura 6 – *Página /docs com todas as rotas agrupadas*

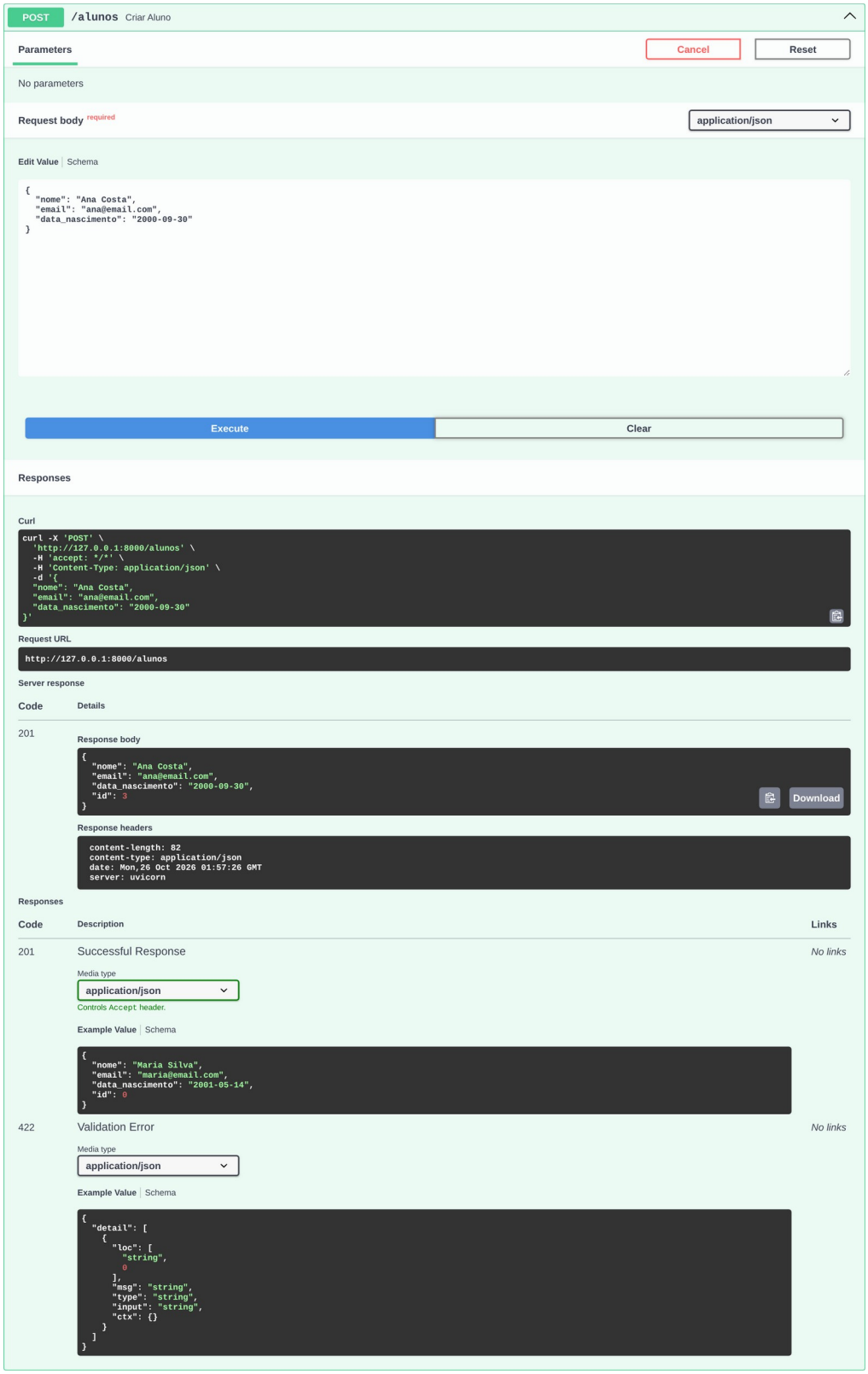


Figura 7 – *Cadastro de aluno pelo Swagger – resposta 201 Created*

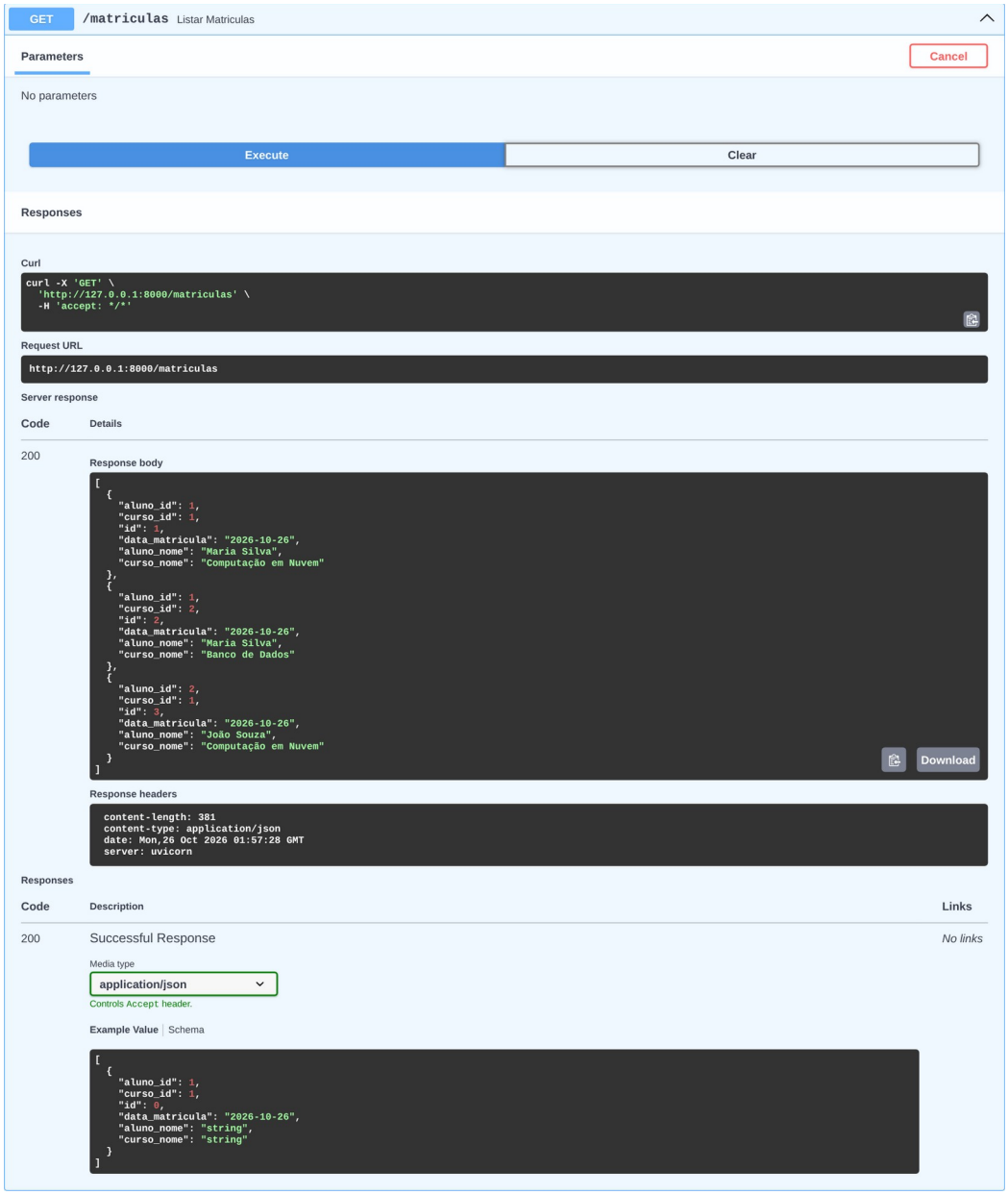


Figura 8 – *Listagem de matrículas pelo Swagger com os nomes do aluno e do curso*



{"status":"ok","banco":"sqlite","horario_utc":"2026-10-26T01:57:28.816736+00:00"}

Figura 9 – *Rota /health aberta no navegador*

6.5 API publicada na Azure

Depois do deploy, a API passou a responder pela internet no endereço **http://52.230.164.119**. A página `/docs` foi aberta no navegador diretamente pelo IP público da VM:

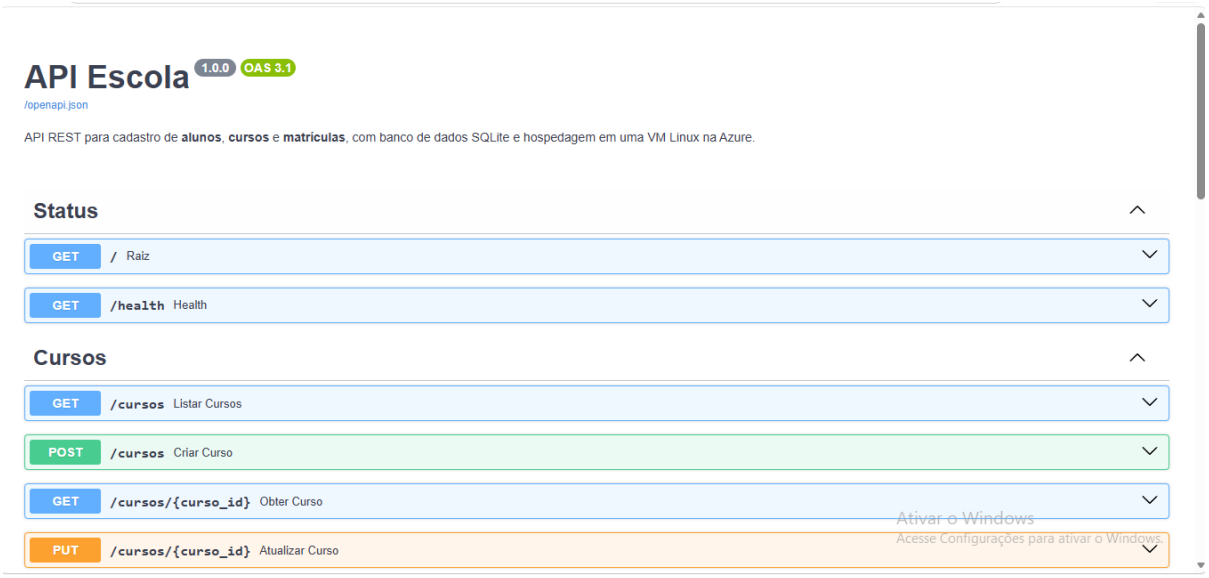


Figura 10 – *Swagger servido pela VM na Azure (http://52.230.164.119/docs)*

O `smoke_test.sh` foi executado no Azure Cloud Shell contra o IP público. Nessa execução, os cadastros retornaram **"E-mail já cadastrado"** e **"Aluno já matriculado"**, porque os mesmos dados tinham sido gravados em uma execução anterior do teste. Isso comprova que o **SQLite persistiu os dados no disco da VM**. A consulta `/alunos/1/cursos` devolveu os dois cursos da aluna, e os erros 409 (matrícula duplicada) e 404 (aluno inexistente) foram retornados corretamente.

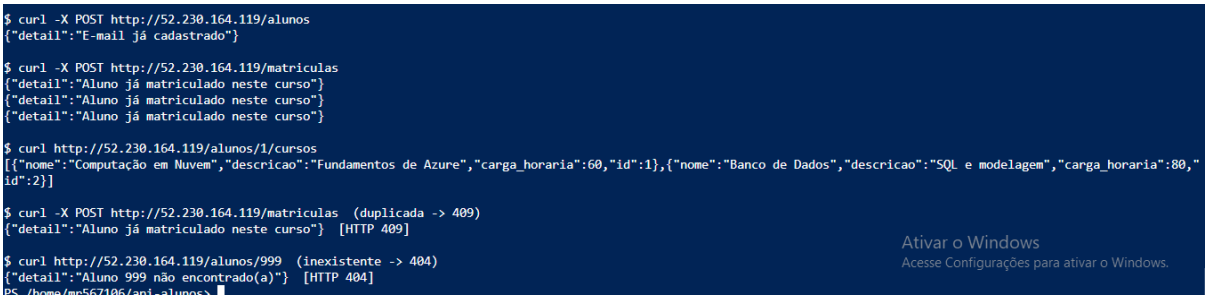


Figura 11 – *smoke_test.sh executado contra a API na Azure*

7. Elementos de nuvem (Microsoft Azure)

Todos os recursos foram criados por um único script (`deploy/azure_criar_vm.sh`) usando a **Azure CLI**, executado no **Azure Cloud Shell**. A conta usada é a conta acadêmica da FIAP, na assinatura **Azure for Students**. Esta seção explica cada elemento, com os valores reais do ambiente publicado.

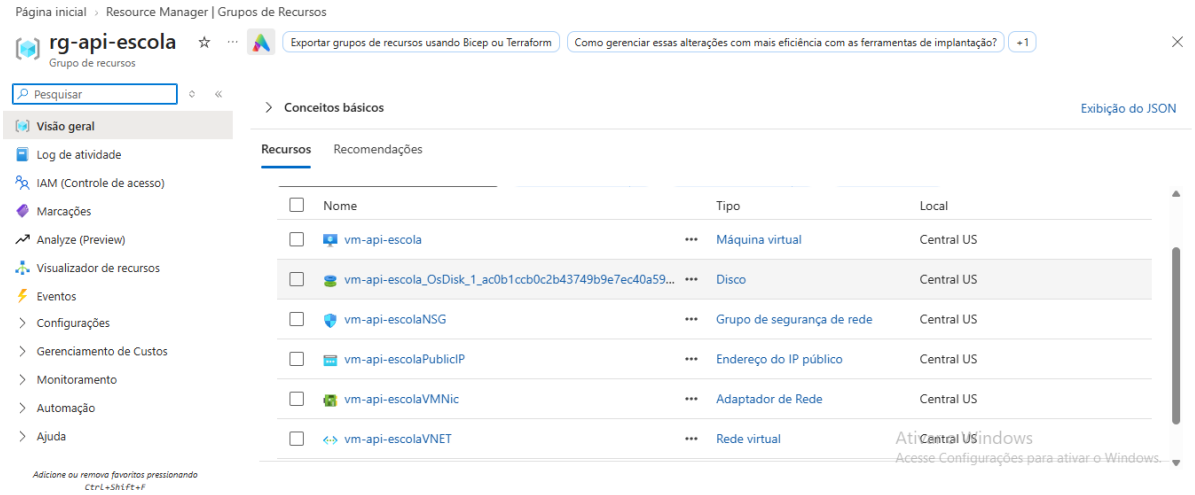


Figura 12 – *Resource Group rg-api-escola com os 6 recursos criados em Central US*

Recurso	Nome / configuração real	Para que serve
Resource Group	rg-api-escola	Agrupa todos os recursos do projeto; apagá-lo remove tudo de uma vez.
Região	Central US (EUA)	Datacenter onde a VM roda. Escolhida porque a assinatura não tinha tamanhos disponíveis em Brazil South (ver 8.6).
Máquina virtual	vm-api-escola (Geração 2)	Servidor Linux que executa Nginx, API e banco.
SKU (tamanho)	Standard_B2pts_v2	2 vCPUs, 1 GiB de RAM, processador Arm64, família B (burstable).
Imagem / SO	Canonical ubuntu-24_04-lts, plano server-arm64	Ubuntu Server 24.04 LTS para Arm64.
Disco do SO	vm-api-escola_OsDisk_1_…, Standard SSD LRS, 30 GiB	Armazena o sistema, a aplicação e o arquivo SQLite.
Rede virtual	vm-api-escolaVNET / vm-api-escolaSubnet	Rede privada onde a VM fica (10.0.0.0/16, sub-rede 10.0.0.0/24).
Interface de rede	vm-api-escolaVMNic	Liga a VM à sub-rede (IP privado 10.0.0.4) e ao IP público.
IP público	vm-api-escolaPublicIP – 52.230.164.119	Endereço pelo qual a API é acessada na internet.
NSG	vm-api-escolaNSG	Firewall: só libera SSH (22) e HTTP

Recurso	Nome / configuração real	Para que serve
		(80).
Autenticação	Chave SSH (sem senha)	Acesso administrativo seguro à VM.

7.1 Máquina virtual e SKU

Na Azure, o **SKU** (ou *size*) define o hardware virtual da VM. O tamanho usado foi o **Standard_B2pts_v2**, confirmado no portal:

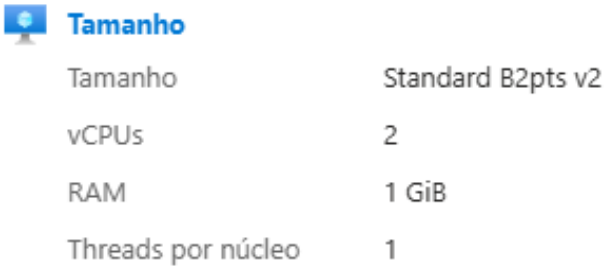


Figura 13 – *Tamanho da VM no portal: Standard B2pts v2, 2 vCPUs, 1 GiB de RAM*

O nome segue a convenção de nomes da Azure e se lê assim:

- **Standard:** camada de preço padrão.
- **B:** família *burstable*. A VM acumula créditos de CPU quando está ociosa e os gasta em picos de uso. É ideal para APIs com tráfego baixo e irregular, como esta.
- **2:** duas vCPUs.
- **p:** processador **Arm** (arquitetura Arm64), mais econômico em energia e custo.
- **t:** *tiny memory*, ou seja, pouca memória por vCPU (1 GiB no total).
- **s:** suporta discos **Premium SSD** (o projeto usa Standard SSD para economizar).
- **v2:** segunda versão dessa série de tamanhos.

Característica	Standard_B2pts_v2
vCPUs	2
Memória RAM	1 GiB
Threads por núcleo	1
Arquitetura	Arm64
Modelo de CPU	Burstable (créditos de CPU)
Uso neste projeto	Nginx + Uvicorn + SQLite consomem bem menos de 1 GiB de RAM em repouso

Como a VM é **Arm64**, foi usada a imagem do Ubuntu específica para Arm (`server-arm64`). Python, Nginx e SQLite funcionam da mesma forma nas duas arquiteturas, então o código da API não precisou de nenhuma mudança. O script de deploy detecta tamanhos Arm (letra **p** no nome) e escolhe a imagem certa automaticamente.

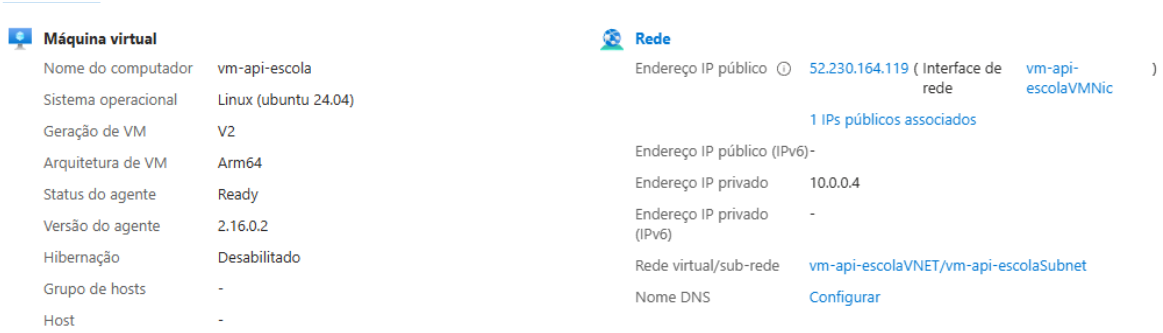


Figura 14 – *Visão geral da VM: Ubuntu 24.04, Geração 2, arquitetura Arm64, IP público e privado*

O tamanho previsto inicialmente era o `Standard_B1s` (x64), mas ele estava bloqueado para a assinatura em Brazil South e nas principais regiões dos EUA. Em **Central US**, os tamanhos liberados eram todos Arm, e entre eles o **B2pts v2** é um dos três tamanhos (junto com B1s e B2ats v2) incluídos nas **750 horas mensais gratuitas** da conta free da Azure para novos clientes, durante 12 meses.

> **Custos:** o disco, o IP público e o tráfego de saída podem ser cobrados à parte. Confirme os valores na Calculadora de Preços da Azure e apague o Resource Group ao final do trabalho com `az group delete --name rg-api-escola --yes`.

7.2 Armazenamento

O armazenamento usado é o **Managed Disk** do sistema operacional (`vm-api-escola_OsDisk_1_…`), um disco virtual gerenciado pela Azure e conectado à VM. Não foram criados discos de dados: o banco SQLite fica no próprio disco do SO. As opções escolhidas no script foram:

- **Tipo Standard SSD:** latência estável e custo menor que o Premium SSD. Atende bem a um banco SQLite pequeno.
- **Redundância LRS (Locally Redundant Storage):** a Azure mantém **3 cópias** síncronas dos dados dentro do mesmo datacenter, protegendo contra falha de hardware.
- **Tamanho 30 GiB:** suficiente para o Ubuntu, Python, Nginx, a aplicação e o banco.
- **Criptografia em repouso:** todos os Managed Disks são criptografados automaticamente pela Azure (SSE com chaves gerenciadas pela plataforma). A criptografia adicional no host e o Azure Disk Encryption ficaram desabilitados, como mostra o print.

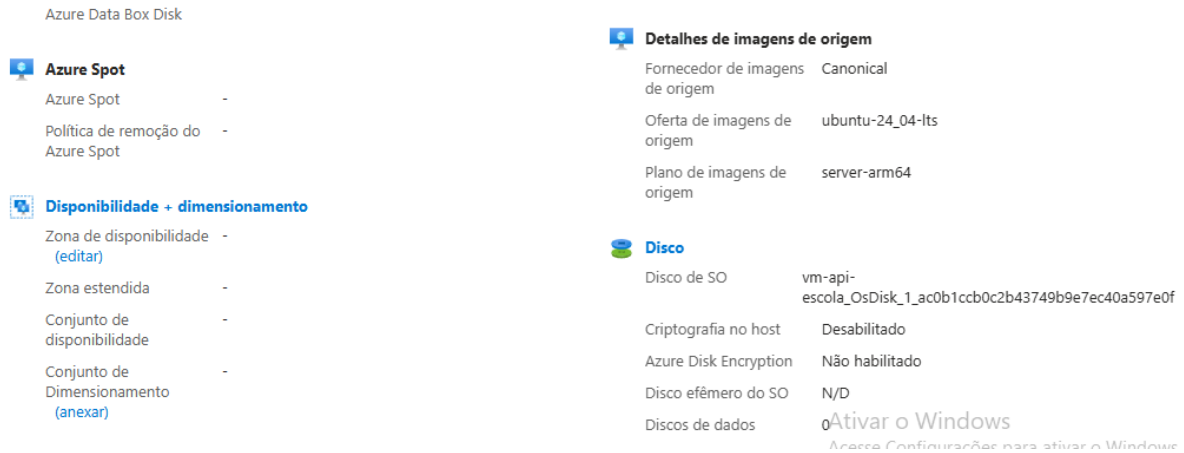


Figura 15 – *Imagem de origem (Canonical, Ubuntu 24.04, server-arm64) e disco do SO*

O arquivo do banco fica em `/var/lib/api-escola/escola.db`, separado do código (`/opt/api-escola`). Assim, atualizar a aplicação com `git pull` não apaga os dados, como comprovado pelo teste da seção 6.5.

7.3 Rede e segurança

A VM tem um adaptador de rede (`vm-api-escolaVMNic`) ligado à sub-rede `vm-api-escolaSubnet`, com **IP privado 10.0.0.4** e **IP público 52.230.164.119**. O tráfego de entrada é filtrado pelo NSG `vm-api-escolaNSG`, associado ao adaptador:

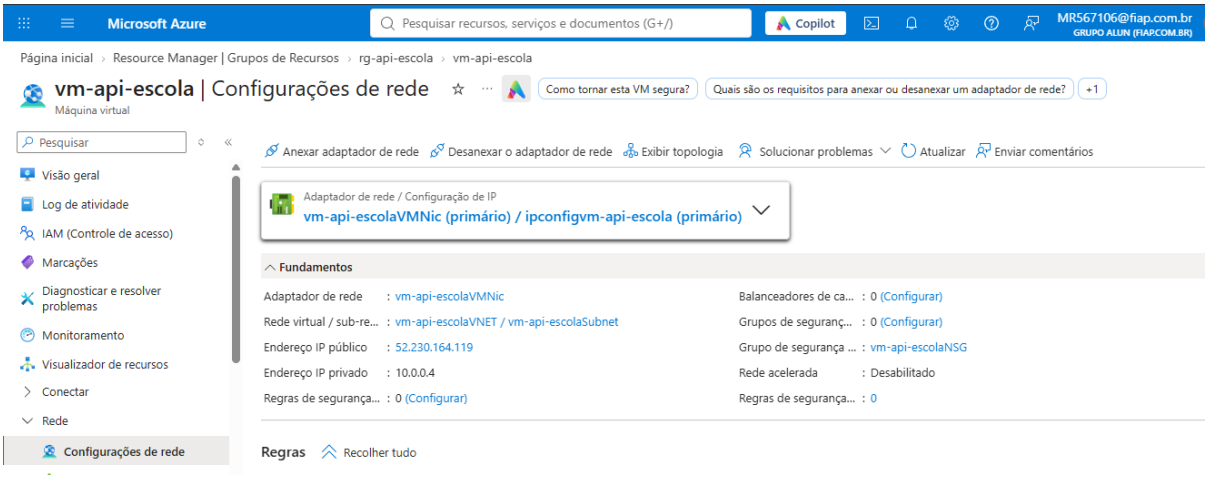


Figura 16 – *Configurações de rede da VM: adaptador, VNet/sub-rede, IPs e NSG*

As regras de entrada do NSG, conferidas no portal, são estas. Segundo o portal, o NSG está associado a **1 interface de rede** (a da VM) e pertence à assinatura **Azure for Students**:

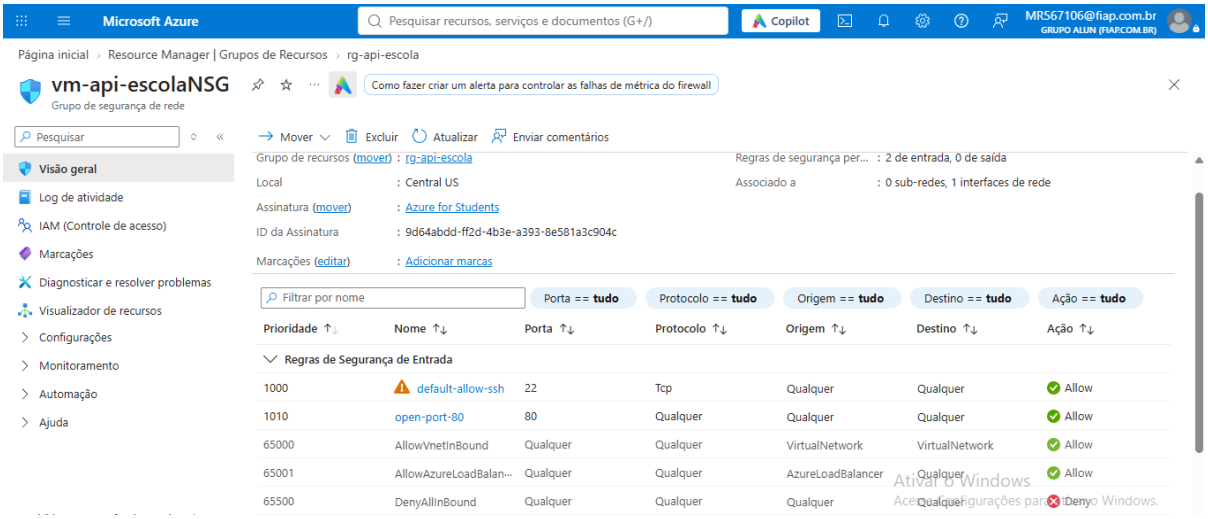


Figura 17 – *Regras de segurança de entrada do NSG vm-api-escolaNSG*

Prioridade	Regra	Porta	Origem → Destino	Ação	Criada por
1000	default-allow-ssh	22/ TCP	Qualquer → Qualquer	Permitir	az vm create
1010	open-port-80	80	Qualquer → Qualquer	Permitir	az vm open-port
65000	AllowVnetInBound	Todas	VirtualNetwork → VirtualNetwork	Permitir	Padrão da Azure
65001	AllowAzureLoadBalancerInBound	Todas	AzureLoadBalancer → Qualquer	Permitir	Padrão da Azure
65500	DenyAllInBound	Todas	Qualquer → Qualquer	Negar	Padrão da Azure

As regras são avaliadas da **menor para a maior prioridade**, e a primeira que combinar com o tráfego decide. Assim, as portas 22 e 80 são liberadas antes de chegar à regra 65500, que **bloqueia todo o resto**. O portal mostra um **ícone de alerta** na regra SSH porque ela está aberta para qualquer origem da internet.

- O **Uvicorn escuta apenas em 127.0.0.1:8000**, então não é acessível de fora; só o Nginx fala com ele.
- O serviço roda com o usuário **www-data**, sem privilégios de administrador.
- O acesso administrativo é feito por **chave SSH**; não há senha para ser descoberta por força bruta.
- **Recomendação:** restringir a regra SSH ao IP do administrador, o que também remove o alerta do portal, e adicionar HTTPS (Let's Encrypt) com um nome DNS.

7.4 Configuração da VM (cloud-init e instalar_na_vm.sh)

O **cloud-init** é um padrão para configurar VMs Linux no primeiro boot. O arquivo `deploy/cloud-init.yaml` é enviado pela opção `--custom-data`. O mesmo processo também está no script `deploy/instalar_na_vm.sh`, que pode ser executado a qualquer

momento pelo **Run Command** da Azure. Foi esse script que configurou a VM neste deploy (ver 8.6). Os dois fazem, sem intervenção manual:

1. instalação de `python3-venv`, `nginx`, `git` e `sqlite3`;
2. `git clone` do repositório do GitHub em `/opt/api-escola`;
3. criação de um ambiente virtual Python e instalação das dependências (`requirements.txt`);
4. criação do serviço **systemd** `api-escola` (inicia no boot e reinicia em caso de falha);
5. configuração do **Nginx** como proxy reverso da porta 80 para a 8000.

Serviço systemd

```
[Service]
User=www-data
WorkingDirectory=/opt/api-escola
Environment=DB_PATH=/var/lib/api-escola/escola.db
ExecStart=/opt/api-escola/.venv/bin/uvicorn app.main:app --host 127.0.0.1 --port 8000
Restart=always
```

Nginx (proxy reverso)

```
server {
    listen 80 default_server;
    location / {
        proxy_pass http://127.0.0.1:8000;
        proxy_set_header Host $host;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
    }
}
```

8. Passo a passo do deploy

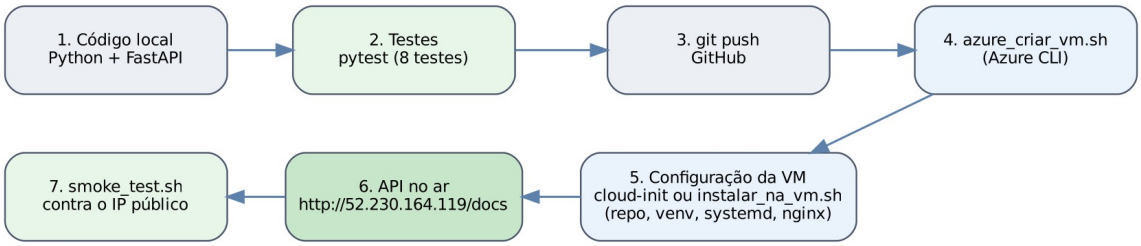


Figura 18 – *Fluxo completo: do código à API publicada*

8.1 Rodar e testar localmente

```
git clone https://github.com/RenesFernandes7366/api-alunos.git
cd api-alunos
python3 -m venv .venv && source .venv/bin/activate
pip install -r requirements-dev.txt
pytest -v                      # 8 testes
uvicorn app.main:app --reload  # http://127.0.0.1:8000/docs
```

8.2 Publicar no GitHub

```
git remote add origin https://github.com/RenesFernandes7366/api-alunos.git
git push -u origin main
```

O repositório precisa ser **público** (ou a VM precisaria de credenciais) para que a VM consiga cloná-lo.

8.3 Criar a infraestrutura na Azure

No portal da Azure, abra o **Cloud Shell**, que já tem a Azure CLI e o login prontos. Comandos usados neste deploy:

```
git clone https://github.com/RenesFernandes7366/api-alunos.git
cd api-alunos
bash -c "LOCATION=centralus SIZE=Standard_B2pts_v2 ./deploy/azure_criar_vm.sh"
```

O script confere se o tamanho está liberado na região, cria o Resource Group, a VM com rede, IP público, NSG e disco, libera a porta 80 e mostra o IP. O `bash -c` faz o comando funcionar tanto no Cloud Shell em **Bash** quanto em **PowerShell**.

8.4 Configurar a VM (quando necessário)

```
az vm run-command invoke -g rg-api-escola -n vm-api-escola \
   --command-id RunShellScript --scripts "@deploy/instalar_na_vm.sh" \
   --query "value[0].message" -o tsv
```

Instala ou reinstala a API dentro da VM e termina mostrando o estado dos serviços e a resposta de `/health`. Também serve para **atualizar** a API depois de um novo `git push`, porque o script faz `git pull` e reinicia o serviço.

8.5 Validar e remover

```
curl http://52.230.164.119/health
bash -c "./scripts/smoke_test.sh http://52.230.164.119"

# ao final do trabalho (para a cobrança):
az group delete --name rg-api-escola --yes --no-wait
```

8.6 Problemas encontrados e soluções

O deploy real passou por cinco problemas, todos resolvidos e incorporados aos scripts do repositório:

Problema	Causa	Solução
Erro da Azure CLI: “The content for this response was already consumed”	Bug da CLI que esconde o erro real. A causa era o tamanho B1s indisponível em Brazil South para a assinatura.	O script passou a verificar a disponibilidade do tamanho (`az vm list-skus`) antes de criar a VM e a mostrar o erro real em caso de falha.
Nenhum tamanho da série B liberado em Brazil South, East US e East US 2	Restrições da assinatura acadêmica por região.	Busca nas principais regiões. Em Central US havia tamanhos Arm liberados; foi escolhido o `Standard_B2pts_v2`, coberto pela conta gratuita.
Imagem incompatível com VM Arm	A imagem padrão do Ubuntu é x64.	O script detecta tamanhos Arm e usa a imagem `ubuntu-24_04-lts:server-arm64`.
`LOCATION=... ./script` não reconhecido	O Cloud Shell estava em PowerShell, que usa outra sintaxe para variáveis.	Uso de `bash -c "..."`, que funciona nos dois shells.
Navegador com ERR_TIMED_OUT; API e Nginx inativos na VM	O cloud-init terminou (`status: done`) sem aplicar a configuração.	Criado o `instalar_na_vm.sh`, executado pelo Run Command da Azure. A API respondeu logo em seguida.

9. Repositório no GitHub

Repositório: https://github.com/RenesFernandes7366/api-alunos

API publicada: http://52.230.164.119/docs

O repositório contém o código, os testes, os scripts de deploy (`azure_criar_vm.sh`, `cloud-init.yaml`, `instalar_na_vm.sh`), o README com instruções e a pasta `docs/` com os diagramas, os 14 prints e este documento em Word e PDF.

```
api-alunos/
├── app/                    # código da API
├── tests/                  # testes automatizados
├── scripts/smoke_test.sh   # teste de ponta a ponta
├── deploy/                 # scripts de infraestrutura e configuração
└── docs/
    ├── diagramas/          # arquitetura, ER, fluxos
    ├── prints/             # 14 prints (local e Azure)
    ├── Documentacao_API_Escola.docx
    └── Documentacao_API_Escola.pdf
```

10. Melhorias futuras e conclusão

10.1 Melhorias possíveis

- **HTTPS** com certificado gratuito Let's Encrypt (Certbot) e um nome DNS para o IP público.
- **Backup do SQLite** com um agendamento (cron) copiando o arquivo para o **Azure Blob Storage**.
- **Autenticação** (token JWT ou chave de API) para proteger as rotas de escrita.
- **CI/CD** com GitHub Actions: rodar os testes a cada push e atualizar a VM automaticamente.
- Para muitos acessos simultâneos de escrita, migrar o banco para o **Azure Database for PostgreSQL**.

10.2 Conclusão

O projeto entrega uma API REST funcional, testada e documentada, publicada em infraestrutura de nuvem real e acessível em **http://52.230.164.119/docs**. O código foi validado por 8 testes automatizados e por chamadas HTTP de ponta a ponta, tanto localmente quanto contra a VM na Azure, onde a persistência dos dados no SQLite também foi comprovada.

Toda a infraestrutura é criada por script, o que torna o processo **reproduzível**. Os problemas encontrados no deploy real (restrições de região e tamanho, VM Arm, diferenças entre PowerShell e Bash e a configuração da VM) foram resolvidos e incorporados aos scripts, deixando o processo mais robusto para as próximas execuções.